Halo 앱 구조 및 초기화 과정 분석

개요

Halo는 Omni 네트워크의 핵심 합의 클라이언트로, Cosmos SDK를 기반으로 구축되었습니다. 이 문서는 Halo 애플리케이션의 구조, 키퍼(Keeper) 시스템, 그리고 초기화 과정을 상세히 분석합니다.

App 구조체 분석

핵심 App 구조체

```
type App struct {
*runtime.App

appCodec codec.Codec
txConfig client.TxConfig
interfaceRegistry codectypes.InterfaceRegistry

// keepers - 각 모듈의 상태 관리자들
AccountKeeper authkeeper.AccountKeeper
BankKeeper bankkeeper.Keeper
StakingKeeper *stakingkeeper.Keeper
SlashingKeeper slashingkeeper.Keeper
```

```
DistrKeeper distrkeeper.Keeper

ConsensusParamsKeeper consensuskeeper.Keeper

EVMEngKeeper *evmengkeeper.Keeper // EVM 엔진 통합

AttestKeeper *attestkeeper.Keeper // 크로스체인 증명 관리

ValSyncKeeper *valsynckeeper.Keeper // 검증자 동기화

EVMStakingKeeper *evmstakinkeeper.Keeper // EVM 스테이킹

RegistryKeeper registrykeeper.Keeper // 체인 레지스트리

EvidenceKeeper evidencekeeper.Keeper

UpgradeKeeper *upgradekeeper.Keeper

MintKeeper mintkeeper.Keeper

EVMRedenomKeeper *evmredenomkeeper.Keeper // EVM 토큰 재명명



// 이벤트 프로세서들 - EVM 이벤트 처리

SlashingEventProc evmslashing.EventProcessor

UpgradeEventProc evmupgrade.EventProcessor

DistributionEventProc evmdistribution.EventProcessor

}
```

키퍼(Keeper) 시스템

Halo는 Cosmos SDK의 키퍼 패턴을 사용하여 각 모듈의 상태를 관리합니다:

1. 표준 Cosmos SDK 키퍼들

- **AccountKeeper**: 계정 관리 및 인증

- **BankKeeper**: 토큰 전송 및 잔액 관리
- **StakingKeeper**: 검증자 스테이킹 관리
- **SlashingKeeper**: 검증자 슬래싱 처리
- **DistrKeeper**: 보상 분배 관리

2. Omni 특화 키퍼들

- **EVMEngKeeper**: EVM 엔진과의 통합 관리
- **AttestKeeper**: 크로스체인 증명(attestation) 처리
- **ValSyncKeeper**: 검증자 세트 동기화
- **RegistryKeeper**: 체인 레지스트리 관리
- **EVMStakingKeeper**: EVM 기반 스테이킹 처리

의존성 주입(Dependency Injection) 시스템

모듈 구성

Halo는 `depinject` 패키지를 사용하여 의존성을 관리합니다:

```
func newApp(
ctx context.Context,
network netconf.ID,
logger sdklog.Logger,
db dbm.DB,
engineCl ethclient.EngineClient,
voter atypes.Voter,
chainVerNamer atypes.ChainVerNameFunc,
```

```
chainNamer rtypes.ChainNameFunc,
feeRecProvider etypes.FeeRecipientProvider,
evmRedenomSubmitConfig evmredenomsubmit.Config,
appOpts servertypes.AppOptions,
asyncAbort  chan<- error,
baseAppOpts ...func(*baseapp.BaseApp),
) (*App, error)
```

모듈 초기화 순서

```
genesisModuleOrder = []string{
authtypes.ModuleName, // 인증 모듈
banktypes.ModuleName, // 은행 모듈
distrtypes.ModuleName, // 분배 모듈
stakingtypes.ModuleName, // 스테이킹 모듈
slashingtypes.ModuleName, // 슬래싱 모듈
minttypes.ModuleName, // 민팅 모듈
genutiltypes.ModuleName, // 제네시스 유틸리티
evidencetypes.ModuleName, // 증거 모듈
upgradetypes.ModuleName, // 업그레이드 모듈
valsynctypes.ModuleName, // 검증자 동기화
engevmtypes.ModuleName, // EVM 엔진
```

```
}
```

Cosmos SDK 통합 방식

1. 런타임 통합

Halo는 Cosmos SDK v0.50의 새로운 런타임 시스템을 사용합니다:

```
type App struct {
*runtime.App // Cosmos SDK 런타임 임베딩
// ... 기타 필드들
}
```

2. 모듈 관리자

각 모듈은 `BeginBlocker`, `EndBlocker` 순서로 실행됩니다:

```
beginBlockers = func(context.Context) []string {
return []string{
minttypes.ModuleName, // 민팅이 먼저 실행
distrtypes.ModuleName, // 분배 처리
slashingtypes.ModuleName, // 슬래싱 처리
stakingtypes.ModuleName, // 스테이킹 처리
evidencetypes.ModuleName, // 증거 처리
attesttypes.ModuleName, // 증명 처리
}
```

```
}

endBlockers = func(context.Context) []string {
return []string{
attesttypes.ModuleName, // 증명 완료
evmstakingtypes.ModuleName, // EVM 스테이킹 완료
valsynctypes.ModuleName, // 검증자 동기화 (스테이킹 래핑)
upgradetypes.ModuleName, // 업그레이드 처리
}
}
```

3. 키퍼 간 의존성 설정

```
// 키퍼 간 의존성 연결
app.EVMEngKeeper.SetVoteProvider(app.AttestKeeper)
app.AttestKeeper.SetValidatorProvider(app.ValSyncKeeper)
app.AttestKeeper.SetPortalRegistry(app.RegistryKeeper)
```

초기화 과정 분석

1. 애플리케이션 생성 단계

```
func  newApp(...) (*App, error) {
```

```go
// 1. 의존성 주입 설정
depCfg := depinject.Configs(
depinject.Provide(diProviders...),
depinject.Invoke(diInvokers...),
depinject.Configs(bankWrapperBindings...),
appConfig(ctx, network),
depinject.Supply(...),
)

// 2. 앱 구조체 초기화
var app = new(App)
var appBuilder = new(runtime.AppBuilder)

// 3. 의존성 주입 실행
if err := depinject.Inject(depCfg, dependencies...); err != nil {
return nil, errors.Wrap(err, "dep inject")
}

// 4. 키퍼 간 의존성 설정
app.EVMEngKeeper.SetVoteProvider(app.AttestKeeper)
// ...

// 5. BaseApp 옵션 설정
```

```
baseAppOpts = append(baseAppOpts, func(bapp *baseapp.BaseApp) {
bapp.SetPrepareProposal(app.EVMEngKeeper.PrepareProposal)
bapp.SetProcessProposal(makeProcessProposalHandler(...))
bapp.SetExtendVoteHandler(app.AttestKeeper.ExtendVote)
bapp.SetVerifyVoteExtensionHandler(app.AttestKeeper.VerifyVoteExtension)
})

// 6. 앱 빌드
app.App = appBuilder.Build(db, nil, baseAppOpts...)

return  app, nil
}
```

2. ABCI 핸들러 설정

Halo는 ABCI++ 인터페이스를 구현하여 CometBFT와 통합됩니다:

```
// PrepareProposal: EVM 엔진이 새 블록 제안 생성
bapp.SetPrepareProposal(app.EVMEngKeeper.PrepareProposal)

// ProcessProposal: 제안된 블록 검증
bapp.SetProcessProposal(makeProcessProposalHandler(makeProcessProposalRouter(app

// ExtendVote: 증명 키퍼가 투표 확장
```

```
bapp.SetExtendVoteHandler(app.AttestKeeper.ExtendVote)


// VerifyVoteExtension: 투표 확장 검증
bapp.SetVerifyVoteExtensionHandler(app.AttestKeeper.VerifyVoteExtension)
```

3. 업그레이드 핸들러 설정

```
func (a App) setUpgradeHandlers(ctx context.Context) error {
// 모든 업그레이드 핸들러 등록
for _, u := range upgrades.Upgrades {
a.UpgradeKeeper.SetUpgradeHandler(u.Name, u.HandlerFunc(a))
}


// 디스크에서 업그레이드 정보 읽기
upgradeInfo, err := a.UpgradeKeeper.ReadUpgradeInfoFromDisk()
if err != nil {
return errors.Wrap(err, "read upgrade info from disk")
}


// 스토어 로더 설정
for _, u := range upgrades.Upgrades {
if u.Name == upgradeInfo.Name {
a.SetStoreLoader(upgradetypes.UpgradeStoreLoader(upgradeInfo.Height, u.Store(ctx
```

```
return nil
}
}

return nil
}
```

모듈별 특화 기능

1. EVM 엔진 통합

```
type Keeper struct {
engineCl ethclient.EngineClient // Geth 엔진 클라이언트
voteProvider types.VoteExtensionProvider
eventProcs []types.EvmEventProcessor
cmtAPI comet.API
feeRecProvider types.FeeRecipientProvider
maxWithdrawalsPerBlock uint64
// ...
}
```

2. 크로스체인 증명 시스템

AttestKeeper는 크로스체인 메시지의 증명을 관리합니다:

- 투표 확장을 통한 증명 수집
- 검증자 서명 검증
- 크로스체인 메시지 승인

3. 검증자 동기화

ValSyncKeeper는 EVM과 Cosmos 검증자 세트를 동기화합니다:

- 스테이킹 모듈 래핑
- EVM 체인으로 검증자 업데이트 전파

설정 및 네트워크 구성

네트워크별 설정

```
const (
deliverIntervalProtected = 1  // 보호된 네트워크에서 배치 비활성화
deliverIntervalEphemeral = 2  // 테스트 중 빠른 업데이트
)

func deliverInterval(network netconf.ID) int64 {
if network.IsProtected() {
return deliverIntervalProtected
}
return deliverIntervalEphemeral
}
```

모듈 계정 권한

```
moduleAccPerms = []*authmodulev1.ModuleAccountPermission{
{Account: authtypes.FeeCollectorName},
{Account: distrtypes.ModuleName, Permissions: []string{authtypes.Burner}},
{Account: stakingtypes.BondedPoolName, Permissions: []string{authtypes.Burner, a
{Account: evmstakingtypes.ModuleName, Permissions: []string{authtypes.Burner, au
// ...
}
```

요약

Halo 앱 구조는 다음과 같은 특징을 가집니다:

1. **모듈화된 아키텍처**: 각 기능이 독립적인 키퍼로 분리
2. **의존성 주입**: `depinject` 를 통한 깔끔한 의존성 관리
3. **ABCI++ 통합**: CometBFT와의 고급 합의 기능 활용
4. **EVM 통합**: Geth 엔진과의 직접적인 통합
5. **크로스체인 지원**: 증명 시스템을 통한 체인 간 통신
6. **업그레이드 지원**: 무중단 네트워크 업그레이드 기능

이러한 구조를 통해 Halo는 Cosmos SDK의 강력함과 EVM의 호환성을 결합하여 크로스체인 기능을 제공하는 혁신적인 합의 클라이언트를 구현했습니다.